

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code	CSA10 / CSA1023	Course Title	Software Engineering
CO Assessed	CO1	Assessment	Assessment Tool 1 – Case Study Analysis
Weightage	20%	Total Marks	25
Student Name	S. Roshni	Reg. No.	192525185
Date	07-10-2026	Duration	60 minutes

CO1 Statement: Apply advanced methodologies and agile project management techniques to manage software projects effectively

Code of Conduct

I ____S.Roshni (192525185)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____S.Roshni____

Annexure A – Case Study Analysis

Problem Statement 35: Distributed Online Collaboration Platform for Large Student Projects

1. Understand the Problem Statement — 5 Marks

1.1 Background

Software engineering courses increasingly require students to build large, realistic projects in teams of five to fifteen members, sometimes spread across several teams and several locations. Such projects are not just coding exercises. They pass through the full software life cycle: requirements gathering, analysis, design, implementation, testing, documentation, deployment and presentation. Each of these activities produces artefacts that must stay consistent with one another, and each is carried out by different people at different times.

In practice, the tools that students use for these activities are chosen individually and are rarely connected. The result is that a project's real status is known only to the few people who happen to hold the information, and faculty find it hard to judge each student's contribution fairly.

1.2 Problem Explanation

Large student projects involve many students working on different activities such as requirements, design, coding, testing, documentation, and presentations.

When teams work without proper coordination, problems such as duplicated work, merge conflicts, missed deadlines, poor communication, and inconsistent documentation can occur.

Therefore, a Distributed Online Collaboration Platform is proposed to provide a single platform where student teams can manage tasks, requirements, source code, reviews, testing, documentation, and project progress.

In simple words, the problem is the absence of one shared, organised place where a large student team can plan its work, write and review its code, test it automatically, keep its documents in order, and show its progress to the faculty guide. The proposed system must bring these activities together so that everyone sees the same, up-to-date information.

1.3 Objectives

The main objectives are:

- Manage project teams, tasks, milestones, and responsibilities.
- Support collaborative requirements and documentation.
- Integrate Git-based version control and code reviews.
- Provide automated testing and CI/CD.
- Track project progress and risks.
- Support containerized development environments.
- Maintain project history and audit trails.

The table below explains how each objective will be met and how success will be judged.

Objective	How the platform achieves it	Measure of success
Manage teams, tasks and milestones	Task board with assignee, priority, deadline and milestone linkage	Every task has one owner and a due date
Collaborative requirements and documents	Shared editor with version history and review status	Single source of truth for each document
Git-based version control and review	Repository integration, branches, pull requests, review comments	No code merged without a recorded review
Automated testing and CI/CD	Pipeline triggered on every commit or pull request	Build and test result visible within minutes
Track progress and risks	Dashboard with completion %, milestone status and risk register	Faculty can view status without asking the team
Containerized environments	Pre-defined container images for each project	Same setup works on every student's machine
History and audit trails	Time-stamped log of every change and approval	Any change can be traced to a person and time

1.4 Scope

The system covers:

- Team and user management
- Task and issue management
- Requirement management
- Git-based source-code management
- Code review
- Automated testing
- CI/CD
- Documentation
- Containerized development
- Progress dashboards
- Risk tracking
- Audit history

Out of scope

- Developing a new version-control engine (existing Git is integrated, not replaced).
- Real-time video conferencing or a full chat system.
- Automatic grading of student projects (the platform only provides evidence for faculty).
- Hosting of the final deployed student applications beyond the demonstration environment.

1.5 Assumptions and Constraints

- All users have internet access and a modern web browser.
- Students have a basic understanding of Git and software engineering terms.
- The institution can provide server or cloud resources for hosting the platform.
- The platform must be usable by teams of different sizes, from small groups to departments.
- Development of the platform itself is limited by academic time and by the skills of the development team.

2. Key Stakeholders and Challenges — 5 Marks

2.1 Stakeholders and Their Roles

Stakeholder	Role / Responsibility	Expectation from the platform
Students / Developers	Develop code, complete tasks and submit contributions	Clear tasks, easy Git workflow, quick feedback on builds and reviews
Team Leaders	Assign tasks, monitor team progress and manage milestones	Real-time view of progress and workload; early warning of delays
Project Guide / Faculty	Review project progress, requirements and deliverables	Reliable evidence of each student's contribution; summary dashboards
Testers	Perform testing and verify software quality	Test cases linked to requirements; automated reports of failures
Documentation Team	Maintain requirements, reports and project documents	Shared editing, version history and consistent templates
System Administrator	Manage users, roles, permissions and system configuration	Simple user management, secure configuration, monitoring and backups
Reviewer	Review source code and provide feedback	Pull-request view with comments, approval and rejection options

2.2 Detailed Responsibilities

Students / Developers pick up tasks from the board, work on their own branches, commit regularly, raise pull requests, respond to review comments and update the status of their tasks.

Team Leaders break the project into tasks and milestones, balance the workload, hold sprint meetings, remove blockers and report the team's progress to the faculty guide.

Project Guide / Faculty approve the scope and requirements, monitor milestones through the dashboard, review risks, give feedback and assess individual and team contribution.

Testers prepare test cases, run manual tests where automation is not possible, log defects and confirm that fixes work before a milestone is accepted.

Documentation Team maintain the requirement specification, design documents, user manuals and final reports, and make sure they match the latest version of the software.

System Administrator creates accounts, assigns roles, maintains the servers and container infrastructure, takes backups and handles security incidents.

Reviewer checks code for correctness, readability, standards and security, and decides whether a pull request can be merged.

2.3 Major Challenges

- Poor coordination between different teams.
- Duplicated work because tasks are not clearly assigned.
- Merge conflicts when multiple students modify the same code.
- Missed deadlines due to poor progress tracking.
- Inconsistent documentation among teams.
- Difficulty in maintaining code quality.
- Manual testing can be time-consuming.
- Lack of proper project history and accountability.
- Different development environments may cause compatibility problems.
- Unauthorized users may access project information without proper security.

2.4 Challenges Faced by Each Stakeholder

Stakeholder	Main challenges	Related issue
Students / Developers	Unclear task ownership; merge conflicts; setup problems on personal machines	Duplicated work, lost time
Team Leaders	No single view of progress; hard to balance workload; late discovery of delays	Missed deadlines
Project Guide / Faculty	Cannot verify who did what; progress reports are manual and often outdated	Lack of accountability
Testers	Manual testing is slow; requirements and test cases are not linked	Poor quality
Documentation Team	Several versions of the same document; documents differ from the code	Inconsistent documentation
System Administrator	Managing many accounts and permissions; protecting project data	Security risk
Reviewer	Review feedback scattered over chats and e-mails; no record of decisions	Weak code quality control

These challenges are interlinked. For example, unclear task assignment leads to duplicated work, which leads to merge conflicts, which delays milestones. A solution must therefore address coordination, quality and visibility together rather than one at a time.

3. Analyze the Current Approach — 5 Marks

3.1 Existing System / Current Process

Currently, large student projects may use different tools separately for different activities. For example:

- Messaging applications for communication
- Spreadsheets for task tracking
- Git repositories for source code
- Separate documents for requirements
- Manual testing

- Separate cloud storage for reports

These tools may not be properly integrated.

Typical workflow in the current process

1. The team leader creates a spreadsheet listing the tasks and shares it in a messaging group.
2. Students discuss requirements in chat and a team member writes them in a separate document.
3. Each student writes code on a personal machine and pushes it to a Git repository, often directly to the main branch.
4. Testing is done manually by one or two members just before the deadline.
5. Reports and presentations are saved in cloud folders, with several copies of each file.
6. The faculty guide asks for progress in review meetings and receives manual status reports.

3.2 Strengths

- Students can communicate easily.
- Git provides version control.
- Documents can be shared online.
- Teams can work remotely.
- Basic task tracking can be performed.

The tools are also familiar and mostly free, so students need little initial training. These strengths should be retained: the new platform will keep Git as its version-control basis and will remain web-based so that teams can work remotely.

3.3 Limitations

- Information is distributed across multiple platforms.
- Difficult to get a complete view of project progress.
- Tasks may be duplicated.
- Merge conflicts may occur.
- Manual testing requires more time.
- Documentation may become inconsistent.
- Project risks may not be identified early.
- Development environments may differ between students.
- Code reviews may not be properly recorded.
- Project history and evidence may be difficult to maintain.

3.4 Gaps to be Addressed

The proposed system should provide one integrated platform for:

Tasks + Requirements + Git + Code Review + Testing + CI/CD + Documentation + Containers + Dashboard + Audit Trail

Gap	Impact on the project	Remedy in the proposed system
Scattered information	No complete view of status; time wasted searching	Single integrated workspace and dashboard
No task–code–requirement link	Cannot trace a requirement to its code and tests	Traceability between requirements, tasks, commits and tests
Manual testing	Late detection of defects	Automated tests triggered by CI/CD
Unrecorded reviews	Poor code quality and no accountability	Pull-request based review with stored comments

Gap	Impact on the project	Remedy in the proposed system
Environment differences	'Works on my machine' errors	Containerized development environments
No early risk detection	Problems found only near the deadline	Risk register and milestone alerts
No audit history	Contribution cannot be verified	Time-stamped audit trail

3.5 Illustrative Scenario

Consider a team of twelve students building an inventory application. Two students separately implement the login page because the task list in the spreadsheet was not updated. Both push changes to the same file and a merge conflict appears two days before submission. The tester finds that the requirement document in the cloud folder is older than the version the developers used. The faculty guide only learns of the delay in the final review. With an integrated platform, the task would have shown an owner, the conflicting branch would have been caught in review, the requirement would have had a single current version, and the dashboard would have warned the guide of the slipping milestone.

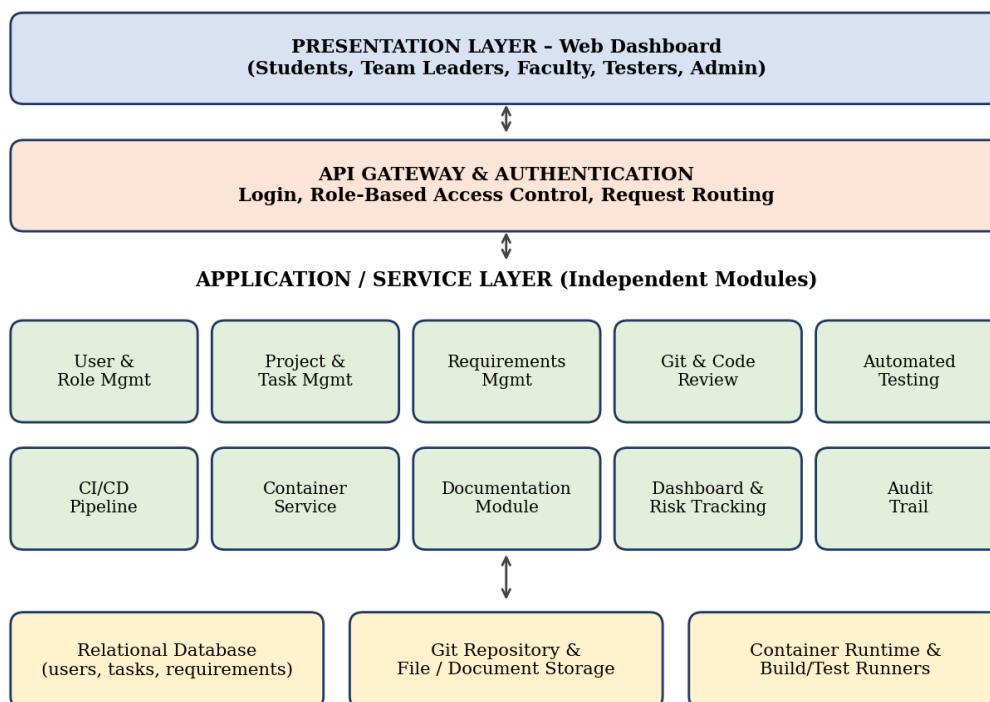
4. Proposed Software Solution & Software Engineering Principles — 5 Marks

4.1 Proposed Solution

A Distributed Online Collaboration Platform will be developed as a web-based platform. It will integrate project management, Git workflows, code review, automated testing, CI/CD, documentation and dashboards.

The platform follows a layered, modular architecture. The presentation layer is a web dashboard used by all roles. An API gateway handles authentication and role-based authorisation. Independent service modules implement each function and communicate through well-defined interfaces. A data layer stores relational data, Git repositories and documents, and a container runtime executes builds, tests and development environments.

Figure 1: Layered architecture of the proposed platform



Suggested technology choices

Component	Possible technology
Front end	Web application using HTML, CSS and a JavaScript framework such as React
Back end / APIs	REST services using Node.js, Java Spring Boot or Python
Database	Relational database such as PostgreSQL or MySQL
Version control	Git, with a hosting service for repositories and pull requests
CI/CD	Pipeline tools such as Jenkins or GitHub Actions
Containers	Docker images, orchestrated by Docker Compose or Kubernetes
Security	JWT or OAuth 2.0 authentication, HTTPS, role-based access control

These are suggestions; the modular design allows any component to be replaced without changing the rest of the system.

4.2 Major Functional Modules

1. User and Role Management

Controls who can enter the platform and what each person can see or change.

- Student registration/login
- Team creation
- Role-based access
- Permission management

2. Project and Task Management

Gives every task one owner, a priority and a deadline, which prevents duplicated work.

- Create projects
- Create and assign tasks
- Set priorities
- Set deadlines
- Manage milestones

3. Requirements Management

Keeps requirements in one place and links each one to tasks, code and test cases.

- Create requirements
- Edit and review requirements
- Track requirement status
- Maintain requirement history

4. Git and Code Review

Ensures that code is merged only after review, reducing conflicts and defects.

- Git repository integration
- Branch management
- Pull requests
- Code reviews
- Review comments

5. Automated Testing

Replaces repeated manual testing with fast, repeatable checks.

- Run test cases automatically
- Generate test results

- Detect failures

6. CI/CD Module

Builds and tests every change so that errors are found early.

- Automatically build the project
- Run tests
- Report build failures
- Support continuous integration and delivery

7. Containerized Development

Gives every student an identical environment from the first day.

- Provide consistent development environments
- Use containers for project setup
- Reduce environment-related problems

8. Documentation Module

Provides shared editing and templates so documents remain consistent.

- Collaborative documentation
- Project reports
- Design documents
- Version history

9. Dashboard and Risk Tracking

Presents progress to students, leaders and faculty in one view.

- Task completion percentage
- Milestone status
- Test results
- Project risks
- Team performance

10. Audit Trail

Records who did what and when, supporting accountability and fair assessment.

- Record project activities
- Track changes
- Maintain project history

4.3 Role-wise Access (Permission Matrix)

Function	Student	Leader	Faculty	Tester	Doc Team	Reviewer	Admin
Manage users and roles	–	–	–	–	–	–	Yes
Create / assign tasks	–	Yes	View	–	–	–	–
Update own tasks	Yes	Yes	–	Yes	Yes	–	–
Edit requirements	Propose	Yes	Approve	–	Yes	–	–
Push code / raise PR	Yes	Yes	–	–	–	–	–
Approve pull requests	–	Yes	–	–	–	Yes	–
Run tests / view results	Yes	Yes	View	Yes	–	View	–
Edit documents	Yes	Yes	Comment	–	Yes	–	–
View dashboard and audit	Own team	Team	All	Team	Team	Team	All

4.4 Functional Requirements

- **FR-01:** The system shall allow users to create and manage projects.
- **FR-02:** The system shall allow team leaders to assign tasks.
- **FR-03:** The system shall support requirement management.
- **FR-04:** The system shall integrate Git repositories.
- **FR-05:** The system shall support code reviews.
- **FR-06:** The system shall execute automated tests.
- **FR-07:** The system shall support CI/CD pipelines.
- **FR-08:** The system shall provide project dashboards.
- **FR-09:** The system shall support containerized environments.
- **FR-10:** The system shall maintain audit trails.

4.5 Non-Functional Requirements

Requirement	Description	Target / How verified
Security	Protect project data using authentication and authorization.	Encrypted login (HTTPS); role checks on every request; periodic security testing
Scalability	Support multiple teams and projects.	Handle at least several hundred concurrent users by adding servers
Reliability	System should be available and dependable.	Automated backups; failed builds do not corrupt data
Usability	Interface should be simple and easy to use.	A new student can create a task and a pull request after a short orientation
Maintainability	Modules should be easy to update.	Independent modules; coding standards; automated tests
Performance	Dashboard and project information should load quickly.	Pages load within a few seconds under normal load
Availability	Users should be able to access the platform when required.	High availability during project working hours; planned maintenance outside peak time

4.6 Key Data Entities

Entity	Main attributes and relationships
User	ID, name, e-mail, role, team; belongs to one or more teams
Team / Project	ID, name, guide, start and end dates; has many users, tasks and milestones
Task / Issue	ID, title, description, owner, priority, deadline, status; linked to a milestone and requirement
Requirement	ID, description, priority, status, version; linked to tasks and test cases
Repository / Pull Request	Repository URL, branch, commit IDs, reviewer, review comments, approval status
Test Case / Result	ID, linked requirement, expected result, actual result, pass/fail, run time
Build / Pipeline Run	Run ID, trigger, stages, status, logs, time stamp
Document	ID, title, content, version number, author, last-modified time
Risk	ID, description, probability, impact, owner, mitigation, status
Audit Log	Event ID, user, action, object changed, old and new values, time stamp

4.7 CI/CD Pipeline Flow

When a student pushes code or raises a pull request, the platform automatically runs the following stages. The result is posted back to the pull request and to the dashboard.

7. Source: the new commit is fetched from the repository.
8. Build: the application is compiled or packaged inside a container.
9. Test: unit and integration tests are executed and results are stored.
10. Quality check: coding-standard and basic security checks are run.
11. Review gate: the pull request cannot be merged until tests pass and a reviewer approves.
12. Deploy: the approved change is deployed to a demonstration or staging environment.
13. Report: build status, test results and logs are shown on the dashboard and recorded in the audit trail.

4.8 Software Engineering Principles

The following table summarises how each principle is built into the design. Each is then explained.

Principle	What is done	Benefit
Modularity	Ten independent modules with clear interfaces	Parallel development; easy replacement
Scalability	Stateless services, containers, separate data layer	More teams and users can be added
Maintainability	Modular code, coding standards, automated tests, documentation	Faster, safer changes
Reliability	Automated testing, CI/CD, backups, error handling	Fewer failures and faster recovery
Usability	Simple dashboard, role-specific views, consistent layout	Low training effort
Security	Authentication, role-based access, encryption, audit trail	Protection of project data

Modularity

The platform is divided into separate modules such as task management, Git, testing and documentation. Each module has a single responsibility and exposes a clear interface, so a team can work on one module without affecting the others. This also matches the way the work is divided into sprints.

Scalability

The system can support additional teams, projects and users as required. Services are designed to be stateless and are packaged in containers, so more instances can be started when load increases. Data storage is separated from application logic so that it can be expanded independently.

Maintainability

Independent modules make it easier to modify and maintain the system. Coding standards, code reviews, automated regression tests and up-to-date design documents help future developers understand and change the system with confidence.

Reliability

Automated testing and CI/CD help identify errors early. Regular backups, input validation, logging and graceful error handling reduce the chance of data loss or system failure.

Usability

A simple dashboard allows students and faculty to easily understand project status. Each role sees only the options that it needs, and common actions such as creating a task or raising a pull request take few steps.

Security

Role-based access ensures that users can access only the resources permitted to them. Passwords are stored securely, communication uses HTTPS, sessions expire after inactivity, and every important action is written to the audit trail.

5. Justification of Methodology / Framework — 5 Marks

5.1 Selected Methodology: Agile Scrum

Agile Scrum is the most suitable methodology for this project.

Why Scrum is Suitable

- Requirements can change during project development.
- The platform contains many modules that can be developed incrementally.
- Work can be divided into small sprints.
- Each sprint can deliver a working feature.
- Regular reviews help identify problems early.
- Student teams can work collaboratively.
- Feedback from faculty and users can be incorporated continuously.
- Testing can be performed throughout development.
- Risks can be identified and addressed early.
- CI/CD fits well with frequent development and testing.

5.2 Comparison with Other Models

Model	Strengths	Why it is less suitable here
Waterfall	Simple, well documented, clear phases	Requirements are expected to change; feedback comes too late; working software appears only at the end
Spiral	Strong risk analysis, iterative	Heavy risk-management overhead; complex for a student team
Kanban	Visual flow, flexible, low overhead	No fixed iterations or time-boxed goals; weaker planning for milestones
DevOps	Automation, fast delivery, monitoring	Is a culture and toolset rather than a planning model; best used together with Scrum
Agile Scrum	Iterative, incremental, frequent feedback, defined roles	Selected – matches changing requirements, modular delivery and student team structure

5.3 Scrum Roles

- **Product Owner:** Represents project requirements and priorities.
- **Scrum Master:** Coordinates Scrum activities and removes obstacles.
- **Development Team:** Develops, tests and maintains the platform.

In this project the faculty guide or a senior student can act as the Product Owner, a team leader can act as the Scrum Master, and the remaining students form the Development Team.

5.4 Scrum Events and Artefacts

Event / Artefact	Description in this project
Sprint	Time-boxed iteration of two weeks that ends with a working increment
Sprint Planning	Team selects backlog items for the sprint and breaks them into tasks
Daily Scrum	Short daily meeting: what was done, what is planned, what blocks progress
Sprint Review	Demonstration of the increment to the faculty guide and collection of feedback
Sprint Retrospective	Team discusses what went well and what to improve in the next sprint
Product Backlog	Ordered list of all features, requirements and fixes
Sprint Backlog	Tasks chosen for the current sprint, tracked on the task board
Increment	Tested, integrated, working part of the platform

5.5 Example Sprint Plan

Sprint	Major Activities	Expected deliverable
Sprint 1	User, team and project management	Login, roles, team and project creation
Sprint 2	Tasks, issues and milestones	Task board with assignment, priority and deadlines
Sprint 3	Requirements and documentation	Requirement tracking and shared documents with versions
Sprint 4	Git integration and code review	Repository link, branches, pull requests and reviews
Sprint 5	Automated testing and CI/CD	Automatic build and test pipeline with reports
Sprint 6	Containers, dashboard and risk tracking	Container environments, dashboards and risk register
Sprint 7	Integration, testing and deployment	Integrated, tested and deployed release

5.6 Sample Product Backlog (User Stories)

- As a team leader, I want to assign a task to a student so that responsibility is clear.
- As a student, I want to raise a pull request so that my code can be reviewed before merging.
- As a tester, I want test results to appear automatically so that I do not repeat manual checks.
- As a faculty guide, I want a dashboard of milestones and risks so that I can monitor progress without long meetings.
- As a documentation member, I want version history so that I can restore an earlier document.
- As an administrator, I want to assign roles so that users see only what they should.

5.7 Definition of Done

A backlog item is considered complete only when its code is reviewed and merged, automated tests pass in the pipeline, documentation is updated, the feature is demonstrated in the sprint review and no known critical defect remains.

5.8 How Scrum Supports Development and Maintenance

- Development: incremental delivery means that at the end of every sprint a working, tested part of the platform exists.
- Quality: testing and review take place in every sprint rather than at the end.

- **Adaptability:** changes requested by faculty or users are placed in the backlog and handled in the next sprint.
- **Maintenance:** the same cycle of backlog, sprint and release is used for bug fixes and enhancements, and the Definition of Done keeps documentation current.
- **Teamwork:** daily meetings and retrospectives improve communication and shared responsibility.

Thus, Agile Scrum supports continuous development, feedback, testing, collaboration and improvement.

6. Expected Outcomes and Limitations

6.1 Expected Outcomes

The proposed platform will:

- Improve collaboration among student teams.
- Reduce duplicated work.
- Reduce merge conflicts.
- Improve project visibility.
- Improve task and milestone management.
- Strengthen software engineering practices.
- Improve software quality through testing and reviews.
- Support consistent development environments.
- Provide better project-management evidence.
- Maintain complete project history and audit trails.

These outcomes directly match the expected outcomes specified in the case study.

6.2 How the Solution Addresses the Challenges

Identified challenge	How it is addressed
Poor coordination between teams	Shared workspace, task board and dashboard
Duplicated work	Every task has a single owner and visible status
Merge conflicts	Branching, pull requests and regular integration through CI
Missed deadlines	Milestones, deadline tracking and early risk alerts
Inconsistent documentation	Shared documents with templates and version history
Difficulty maintaining code quality	Mandatory code review and automated quality checks
Time-consuming manual testing	Automated tests run on every change
Lack of history and accountability	Audit trail of all activities
Environment compatibility problems	Containerized development environments
Unauthorized access	Authentication and role-based authorization

6.3 Risks and Implementation Challenges

- Integration with Git and CI/CD systems may be complex.
- Students may require training to use the platform.
- Large projects may require significant storage.
- Security vulnerabilities may occur if access control is poorly implemented.
- Container management can require additional resources.
- Network problems can affect online collaboration.

- System maintenance may require technical expertise.

Risk register with mitigation

Risk	Likelihood	Impact	Mitigation
Complex Git / CI/CD integration	Medium	High	Use standard, well-supported tools; build and test integration early (Sprint 4–5)
Low user adoption	Medium	Medium	Short training, simple interface, guidance from faculty
Security weakness	Low	High	Role-based access, regular security tests, secure coding reviews
Storage and resource growth	Medium	Medium	Storage limits, archiving of old projects, scalable hosting
Network or server failure	Low	High	Backups, monitoring, ability to restore quickly
Scope creep	High	Medium	Prioritised backlog and sprint reviews with the Product Owner

6.4 Limitations

- Requires a reliable internet connection.
- Initial development may be time-consuming.
- Students need to learn Git, CI/CD and container tools.
- Hosting and storage may involve additional resources.
- Integration with different development tools can be complex.

6.5 Future Enhancements

- Add AI-based task assignment.
- Add AI-based risk prediction.
- Provide intelligent code-review suggestions.
- Add mobile application support.
- Add advanced project analytics.
- Add automatic documentation generation.
- Add plagiarism/code similarity detection.
- Integrate additional development platforms.

For example, AI-based risk prediction could study past sprint data to warn a team when a milestone is likely to be missed, and a mobile application could send instant notifications about reviews and build failures.

6.6 Conclusion

The Distributed Online Collaboration Platform for Large Student Projects provides an integrated environment for managing requirements, tasks, source code, reviews, testing, documentation and project progress. By using Agile Scrum, Git workflows, CI/CD, automated testing, role-based access and containerized environments, the system can reduce coordination problems and improve software quality.

Therefore, the proposed platform is a scalable, maintainable, secure and reliable solution for managing large student software projects.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria and Marks (for Evaluator)

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25